

FIAP
CURSO DE ENGENHARIA DE SOFTWARE

EDUARDO FELIX FROIS SILVA - RM574103
GABRIEL HENRIQUE ONGARELLI REIS - RM572636
MATHEUS DE AMORIM BRITO - RM572435
THIAGO GOMES NASCIMENTO - RM569436
VINICIUS SCALONE RAMIRES - RM573783

JOVI SMART CAMERA
DOCUMENTAÇÃO DA SPRINT 4

SÃO PAULO
2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 OBJETIVOS.....	3
3 DESCRIÇÃO DO PROJETO.....	4
3.1 Funcionalidades implementadas.....	4
4 ORGANIZAÇÃO DO PROGRAMA.....	5
4.1 Estruturas e validações.....	5
5 ALTERAÇÕES EM RELAÇÃO À SPRINT ANTERIOR.....	6
5.1 Elementos removidos.....	6
5.2 Elementos modificados.....	6
5.3 Elementos adicionados.....	6
6 COMO EXECUTAR.....	8
7 CONCLUSÃO.....	8

1 INTRODUÇÃO

A câmera de um smartphone possui diferentes modos e configurações. Porém, muitos usuários encontram dificuldade para escolher a opção mais adequada em cada situação. O projeto JOVI Smart Camera foi criado para apresentar esses recursos de maneira simples e intuitiva.

A solução consiste em um programa executado no terminal. Por meio de menus, o usuário pode consultar os modos da câmera, cadastrar perfis de configuração, receber recomendações manuais ou baseadas no clima real, simular o registro de fotografias e consultar um histórico que agora permanece salvo entre execuções.

2 OBJETIVOS

O objetivo geral é facilitar a compreensão e a utilização dos recursos da câmera JOVI, aplicando os conceitos estudados na disciplina de Computational Thinking with Python, agora com persistência de dados e integração com uma fonte externa de informação.

Os objetivos específicos são:

- apresentar os modos disponíveis e suas descrições;
- permitir o cadastro e o gerenciamento de perfis;
- recomendar um modo conforme o tipo de cena ou conforme o clima real da cidade do usuário;
- simular fotografias e manter um histórico entre execuções do programa;
- validar as entradas e apresentar mensagens claras ao usuário;
- persistir os dados de perfis e fotografias em arquivos, e consumir uma API externa para enriquecer as recomendações.

3 DESCRIÇÃO DO PROJETO

O JOVI Smart Camera utiliza somente recursos da biblioteca padrão do Python (json e urllib), sem necessidade de instalar bibliotecas externas. Os perfis e o histórico de fotografias, que na Sprint 3 existiam apenas durante a execução, agora são salvos em arquivos JSON e recarregados automaticamente sempre que o programa é aberto.

Os quatro modos disponíveis continuam sendo Retrato, Noturno, Paisagem e Movimento. Cada perfil criado pelo usuário armazena nome, modo, resolução, estado do flash automático e estado da grade de enquadramento.

3.1 Funcionalidades implementadas

- visualizar os modos da câmera;
- cadastrar, consultar, alterar e remover perfis;
- receber uma recomendação de modo conforme a cena, escolhida manualmente;
- receber uma recomendação de modo com base no clima real da cidade digitada, consultando a API pública Open-Meteo;
- simular a captura de uma fotografia;
- consultar o histórico de fotografias, incluindo as de sessões anteriores;
- retornar aos menus após cada funcionalidade;
- encerrar o programa pelo menu principal.

Os perfis e o histórico de fotografias são salvos em arquivos JSON (perfis.json e fotos.json) a cada alteração, e continuam disponíveis mesmo depois que o programa é fechado e reaberto.

4 ORGANIZAÇÃO DO PROGRAMA

O arquivo `main.py` possui 333 linhas e 30 funções. O código foi dividido em blocos responsáveis pela persistência de dados, validação de entradas, exibição de dados, gerenciamento dos perfis, recomendação de modos (manual e por clima), histórico de fotografias e navegação pelos menus.

Cada função de leitura e validação, como `ler_numero` e `ler_sim_nao`, continua sendo usada em todo o programa, evitando repetição de código. A partir do retorno recebido na Sprint 3, as funções de cadastro e alteração de perfil foram reorganizadas: agora existe uma função responsável apenas por coletar os dados digitados pelo usuário, e outra função separada, responsável apenas por aplicar essa informação na lista de perfis e salvar o arquivo. Nenhuma das funções de manipulação de dados realiza leitura de teclado ou impressão na tela. Os comentários do código continuam utilizando o símbolo `#`.

4.1 Estruturas e validações

A lista `MODOS` continua com dicionários de nome, descrição e tipo de cena. A lista de perfis e a lista de fotografias agora também são carregadas a partir dos arquivos `perfis.json` e `fotos.json` no início da execução.

O sistema realiza as seguintes validações:

- números inteiros e intervalos permitidos;
- nomes vazios ou repetidos, inclusive na alteração de um perfil já existente;
- respostas de confirmação diferentes de S ou N;
- tentativas de utilizar listas que ainda estão vazias;
- nome de cidade vazio, na recomendação por clima;
- falhas de conexão, tempo de resposta excedido ou cidade não encontrada, ao consultar a API;
- arquivos de dados ausentes ou corrompidos, tratados na leitura, retornando uma lista vazia em vez de interromper o programa.

Após a execução de uma funcionalidade, o programa retorna ao menu correspondente. A opção `Sair` finaliza o sistema de forma clara.

5 ALTERAÇÕES EM RELAÇÃO À SPRINT ANTERIOR

A Sprint 4 manteve a mesma proposta e a mesma estrutura de menus da Sprint 3, acrescentando persistência de dados, consumo de uma API externa e ajustes na organização interna das funções, conforme o retorno recebido na avaliação anterior.

5.1 Elementos removidos

Nenhuma funcionalidade foi removida da Sprint 3. O que deixou de existir foi o comportamento de os dados durarem apenas uma execução: a mensagem informando que os perfis e o histórico eram apagados ao encerrar o programa não existe mais, porque essa limitação foi eliminada pela persistência em arquivo. Também deixaram de existir, no formato anterior, as versões de `cadastrar_perfil` e `alterar_perfil` que reuniam leitura de dados e manipulação da lista de perfis numa única função; essas duas funções, do jeito que existiam na Sprint 3, foram substituídas pela versão dividida descrita a seguir.

5.2 Elementos modificados

A listagem das resoluções disponíveis era escrita separadamente dentro de `cadastrar_perfil` e dentro de `alterar_perfil`, com o mesmo trecho de código repetido nas duas funções. Essa repetição foi retirada e substituída por uma função própria, `mostrar_resolucoes`, chamada nos dois lugares, no mesmo padrão que já era usado pela função `mostrar_modos`.

As funções `cadastrar_perfil` e `alterar_perfil`, que antes misturavam a leitura de dados do usuário com a alteração direta da lista de perfis, foram reescritas. Cadastrar um perfil passou a ser dividido em duas etapas: `coletar_novo_perfil`, que só faz as perguntas ao usuário e devolve um dicionário com as respostas, e `adicionar_perfil`, que só recebe esse dicionário, insere na lista e salva o arquivo. Alterar um perfil seguiu o mesmo princípio: uma função pergunta qual campo será alterado, outra pergunta o novo valor desse campo, e uma última função, `aplicar_alteracao`, apenas grava esse valor no dicionário do perfil, sem nenhuma leitura de teclado ou impressão na tela.

A recomendação de modo, que na Sprint 3 dependia apenas da escolha manual do tipo de cena, passou a contar com uma segunda opção no menu principal, que consulta o clima real da cidade digitada pelo usuário.

5.3 Elementos adicionados

Foram adicionadas as funções `carregar_perfis`, `salvar_perfis`, `carregar_fotos` e `salvar_fotos`, responsáveis por ler e gravar os dados em arquivos JSON. Essas funções são chamadas no início do programa e sempre que um perfil ou uma fotografia é cadastrado, alterado ou removido.

Foi adicionado o consumo de uma API externa, a Open-Meteo, que não exige chave de acesso. As funções `buscar_coordenadas` e `buscar_clima` fazem as requisições HTTP, `escolher_modos_pelo_clima` decide o modo mais adequado a partir da resposta da API, e `recomendar_modos_clima` organiza essa nova opção do menu principal, incluindo o tratamento de erros de conexão.

Também foi adicionada a função `mostrar_resolucoes`, citada no item 5.2, e um tratamento de exceções mais específico para os novos tipos de erro que passaram a existir com a leitura de arquivos e o acesso à internet.

6 COMO EXECUTAR

Para executar o sistema:

- ter o Python 3 instalado no computador;
- abrir o terminal na pasta do projeto;
- executar o comando: `python main.py`;
- escolher uma das opções apresentadas no menu.

Não é necessário instalar nenhuma biblioteca externa. Para usar a opção de recomendação de modo pelo clima real é necessário estar conectado à internet; sem conexão, o programa mostra um aviso e continua funcionando normalmente.

7 CONCLUSÃO

O JOVI Smart Camera evoluiu, na Sprint 4, de uma solução que existia apenas durante a execução para um programa com persistência de dados e integração com uma API externa, mantendo os menus claros e as validações já existentes na Sprint 3.

A reorganização das funções de cadastro e alteração de perfil, separando a leitura de dados da manipulação da lista de perfis, tornou o código mais alinhado com o retorno recebido na avaliação anterior. A solução continua sem depender de bibliotecas externas para funcionar, exigindo internet apenas para a recomendação de modo baseada no clima.